INVENTORY IMS — Q07: IMPROVE DATA ACCURACY

Student: Dheeraj Khetwal | Session 2026-27 | OVERVIEW & ELEVATOR PITCH

PART 1: 1-PARAGRAPH PROTOTYPE PITCH (ELEVATOR SCRIPT)

Word-for-word spoken pitch to deliver when evaluators ask for an overview.

ELEVATOR SCRIPT — WHAT TO SPEAK TO EVALUATORS

CORE PITCH

"Good morning Sir/Ma'am. Mera project Inventory Management System hai jisme mujhe assigned quality goal Q07: Improve Data Accuracy mila tha. Industry mein lagbhag 60% inventory errors downstream discovery ke time 10x se 100x zyada costly ho jaate hain.

Is problem ko solve karne ke liye maine Python, CustomTkinter aur SQLite (WAL Mode) par ek enterprise desktop application develop ki hai jo Philip Crosby ke 'Zero Defect' aur Shigeo Shingo ke 'Poka-Yoke' principles par based hai.

Hamara system data save hone se pehle hi invalid entries ko 3-tier validation gates par reject karta hai, har stock movement aur update ka immutable JSON audit trail capture karta hai, aur live CTQ Scorecard, Pareto 80/20 Analysis aur FMEA Matrix ke through continuous quality improvement ensure karta hai.

Poora system 42/42 automated pytest tests se verified hai aur code strictly 'Screen Never Writes SQL' clean architecture follow karta hai."

[KEY VIVA TAKEAWAY]:
Yeh paragraph 40-50 seconds mein bolna hai. Isme technology, TQM philosophers (Crosby & Shingo), architecture aur 42/42 tests sab cover ho jaate hain.

CORE TQM ACADEMIC FOUNDATIONS

Key theoretical alignments connecting software to syllabus.

ACADEMIC THEORIES APPLIED IN THIS PROJECT

TQM ROOTS

• Philip Crosby (Zero Defects): Prevention is cheaper than inspection. Fixing errors at input costs 1/10th of post-dispatch corrections.

• Shigeo Shingo (Poka-Yoke): Source inspection at the point of entry. Eliminating inadvertent operator slips before persistence.

• Walter Shewhart & W.E. Deming (SPC): Measuring process variation and applying Plan-Do-Check-Act (PDCA) to software rules.

• Joseph Juran (Pareto 80/20): Concentrating engineering effort on the 'vital few' defect categories causing 80% of data errors.

PART 2: REVIEW 1 SLIDE-WISE EXPLANATION (SLIDES 1 TO 13)

What to speak for each slide in Review 1.

SLIDE 01: TITLE & ACADEMIC IDENTITY

SLIDE 01

Kya bolna hai: 'Sir, yeh mera BBAT104 ka Review 1 & 2 presentation hai. Mera topic hai Enterprise Inventory Management System aur focus goal hai Q07: Improve Data Accuracy. Humne isme core focus Quality at the Gate par rakha hai: Prevent -> Record -> Improve.'

Stack: Python 3.11+, CustomTkinter desktop UI, SQLite3 (WAL mode), Matplotlib SQC charts, pytest.

SLIDE 02: EXECUTIVE SUMMARY & ACADEMIC VISION

SLIDE 02

Kya bolna hai: 'Sir, traditional software mein inspection baad mein hoti hai, jo Philip Crosby ke cost-of-quality rule ke against hai. Agar dispatch ke baad stock error pata chale toh cost 10x se 100x badh jaati hai. Hamari vision hai data enter hote hi defect ko block karna taaki downstream phantom stock aur stockouts 0% ho sakein.'

Key Point: Quality is designed into the workflow, not inspected only at the end.

SLIDE 03: THREE FOUNDATIONAL QUALITY IDEAS

SLIDE 03

Kya bolna hai: 'Humne teen quality pillars integrate kiye hain:

1. Poka-Yoke: Entry par regex, bounds aur controlled dropdowns.

2. Statistical Quality: Pareto 80/20 analysis aur CTQ scorecard.

3. Continuous Auditing: Har change ka structured JSON diff, reason aur timestamp ke sath track hona.'

Cycle: Prevent -> Measure -> Learn -> Prevent Better.

SLIDE 04: PROBLEM STATEMENT & INDUSTRIAL BASELINE

SLIDE 04

Kya bolna hai: 'Sir, manual inventory ka sabse bada risk hai Garbage In, Garbage Out. Users spelling alag likhte hain, negative quantity enter kar dete hain, ya unexplained adjustments karte hain. Isse physical stock aur system stock match nahi hota. Hamara project boundary desktop IMS ke andar data accuracy aur mutation integrity ko enforce karna hai.'

Consequences: Phantom stock, procurement delays, and unreconciled balance sheets.

SLIDE 05: STAKEHOLDER MATRIX & QUALITY NEEDS (VOC)

SLIDE 05

Kya bolna hai: 'TQM mein Voice of Customer (VOC) sabse pehle aati hai:

- Store Manager ko live valuation aur low-stock alert chahiye.
- Procurement Officer ko reliable reorder signals chahiye.
- Auditor ko forensic JSON audit trail chahiye.
- Warehouse Operator ko fast, error-proof input UI chahiye.'

Design Principle: Controls strict hone chahiye data integrity par, par user ke liye clear error message hona chahiye.

SLIDE 06: MEASURABLE QUALITY REQUIREMENTS & VIVA METRICS

SLIDE 06

Kya bolna hai: 'Sir, humne qualitative claims ke sath verifiable metrics set kiye hain:

- SKU Uniqueness: 100% database constraint + service validation.
- Non-negative stock & positive price: 100% enforced.
- Category taxonomy: Free-text hatakar 100% controlled dropdown.
- Regression testing: 42 out of 42 automated tests pass ho rahe hain.'

Viva Note: Alag se baseline (qualitative) aur post-intervention verified results bataye hain.

SLIDE 07: FIVE MEASURABLE QUALITY OBJECTIVES (O1 TO O5)

SLIDE 07

Kya bolna hai: 'Hamare 5 testable objectives hain:

- O1: Entry gate par invalid SKU aur category reject karna.
- O2: Quantity >= 0 aur Price > 0 invariants protect karna.
- O3: Har CREATE, UPDATE, DELETE aur adjustment ko JSON diff ke sath log karna.
- O4: Pareto aur CTQ ke through recurring defects prioritize karna.
- O5: 3-tier maintainable architecture maintain karna.'

SLIDE 08: SYSTEM SCOPE, BOUNDARIES & ROLE-BASED ACCESS

SLIDE 08

Kya bolna hai: 'Sir, project ki boundaries defined hain. In-Scope: Desktop CRUD, Stock adjustments, Audit history aur TQM analytics. Out-of-Scope: Multi-tenant cloud aur RFID hardware. Humne principle of least privilege ke under Admin, Manager aur Staff ke role permissions define kiye hain.'

SLIDE 09: Q07 FEATURE SPECIFICATION — 5 POKA-YOKE ENHANCEMENTS

SLIDE 09

Kya bolna hai: 'Yeh hamare mandatory 5 Poka-Yoke features hain:

- F1 (Validation): SKU regex, positive numbers, whitespace trim.
- F2 (Controlled Taxonomy): 5 fixed categories, duplicate spelling impossible.
- F3 (Debounced Search): 300ms indexed search, accidental duplicate product create hone se rokta hai.
- F4 (Confirmation Modal): Deletion ya adjustment se pehle item details ka preview popup.
- F5 (JSON Audit History): Atomic change log with old/new state.'

SLIDE 10: VIVA MAPPING: FEATURES VS TQM DIMENSIONS

SLIDE 10

Kya bolna hai: 'Sir, humne software features ko David Garvin ke 8 Dimensions (Conformance, Reliability, Serviceability) aur Deming ke 14 Principles se map kiya hai. For example, Deming ka Principle 3 hai Build quality into the process, cease mass inspection—yeh hamara Poka-Yoke input gate directly fulfill karta hai.'

SLIDE 11: END-TO-END SIPOC PROCESS MAPPING

SLIDE 11

Kya bolna hai: 'SIPOC diagram hamari process boundary dikhata hai:

• Suppliers: Vendors aur Warehouse.

• Inputs: SKU, category, purchase orders.

• Process: 5 steps—Validate -> Deduplicate -> Commit Transaction -> Write Audit Diff -> Recalculate CTQ.

• Outputs: Verified records, valuation, alerts.

• Customers: Store Manager, Procurement aur Auditors.'

SLIDE 12: ARCHITECTURAL INTEGRITY — SCREEN NEVER WRITES SQL

SLIDE 12

Kya bolna hai: 'Sir, hamari clean code rule hai: Screen kabhi direct SQL nahi likhegi.

• Presentation Layer (app/views) sirf service ko call karti hai.

• Service Layer (app/services) business rules aur Poka-Yoke validation run karti hai.

• Persistence Layer (app/repositories) safe parameterized SQL execute karti hai. Isse separation of concerns aur testability guarantee hoti hai.'

SLIDE 13: RELATIONAL DATABASE SCHEMA & ACID INTEGRITY

SLIDE 13

Kya bolna hai: 'Hum SQLite ko WAL Mode (Write-Ahead Logging) aur PRAGMA foreign_keys = ON ke sath run karte hain. Database schema mein CHECK(quantity >= 0) aur UNIQUE(sku) lagaya hai. Koi bhi multi-step operation atomic transaction ke andar commit ya rollback hota hai, ensuring ACID compliance.'

PART 3: REVIEW 2 SLIDE-WISE EXPLANATION (SLIDES 14 TO 24)

What to speak for working implementation and quality tools.

SLIDE 14: LIVE WORKING IMPLEMENTATION (DASHBOARD)

SLIDE 14

Kya bolna hai: 'Sir, yeh hamara live CustomTkinter application hai. Dashboard real-time SQLite se data fetch karta hai: yahan 11 unique products, 634 total units, $328,099 ki total valuation aur 3 low-stock alerts hain. Category Valuation chart interactive hai aur bar par hover karte hi exact valuation aur percentage dikhata hai.'

Visuals: 4 KPI cards, Category Valuation Bar Chart, Recent Audit Activity stream.

SLIDE 15: FULL CRUD LIFECYCLE WITH 3-TIER POKA-YOKE GATES

SLIDE 15

Kya bolna hai: 'Yahan product creation aur edit workflow hai. Jaise hi user Add New Product par click karta hai, modal form khulta hai. Category controlled dropdown se aati hai, SKU auto-format hota hai. Agar koi negative quantity ya empty field daalta hai, toh system save hone se pehle clear warning dialog dikhata hai.'

Path: User Input -> UI Validation -> Service Gate -> SQLite Transaction -> Audit Diff.

SLIDE 16: STOCK MOVEMENT & INVENTORY VARIANCE MANAGEMENT

SLIDE 16

Kya bolna hai: 'Inventory tab mein stock valuation ka formula Sum(Quantity * Unit Price) continuously compute hota hai. Stock adjustment ke liye mandatory reason (jaise Damaged Stock ya Audit Reconciliation) select karna padta hai. Unexplained stock adjustments impossible hain.'

Invariants: Quantity strictly non-negative; low stock alerts trigger automatically when Qty <= Reorder Level.

SLIDE 17: IMMUTABLE AUDIT TRAIL & FORENSIC JSON STATE DIFFING

SLIDE 17

Kya bolna hai: 'Sir, yeh hamari forensic audit screen hai. Har row ke aage Inspect JSON button hai. Popup mein hum dekh sakte hain ki purani value kya thi, nayi value kya hui, delta kitna tha, kisne change kiya aur timestamp kya tha. Yeh screen 100% read-only hai, koi bhi user history delete ya edit nahi kar sakta.'

Format: Valid JSON snapshot with old_state, new_state, delta, reason, and UTC timestamp.

SLIDE 18: CRITICAL TO QUALITY (CTQ) PERFORMANCE SCORECARD

SLIDE 18

Kya bolna hai: 'Six Sigma CTQ methodology ke under humne 5 critical parameters define kiye hain: SKU uniqueness, non-negative stock, price validity, category standard aur audit coverage. System live database scan karke 100% compliance confirm karta hai.'

Target: 100% compliance across all 5 checks. Proves zero defect state in database.

INVENTORY IMS — Q07: IMPROVE DATA ACCURACY

Student: Dheeraj Khetwal | Session 2026-27 | REVIEW 1 & 2 VIVA DEFENSE GUIDE

SLIDE 19: STATISTICAL DEFECT PARETO ANALYSIS — 80/20 RULE

SLIDE 19

Kya bolna hai: 'Joseph Juran ke 80/20 rule ke according, humne intercepted entry errors ko Pareto chart par plot kiya hai. Bars descending order mein hain aur red line cumulative percentage dikhati hai. Data prove karta hai ki top 2 causes—Missing Fields aur Invalid Price Format—hi 67.9% defects ke liye responsible hain. Humne Poka-Yoke se sabse pehle inhi Vital Few ko eliminate kiya hai.'

Formula: Cumulative % = (Sum defects 1..k / Total defects) * 100.

SLIDE 20: 4M CAUSE-AND-EFFECT ANALYSIS (ISHIKAWA FISHBONE)

SLIDE 20

Kya bolna hai: 'Inventory inaccuracy ka root cause analyse karne ke liye humne Ishikawa diagram design kiya across 4Ms:

- Man: User fatigue aur lack of training.
- Machine: UI par validation na hona.
- Method: Unrecorded manual adjustments.
- Material: Ghalat SKU labels aur duplicate categories.

Hamare Q07 features in sabhi root causes ko counteract karte hain.'

SLIDE 21: FMEA RISK PRIORITIZATION & RPN OPTIMIZATION

SLIDE 21

Kya bolna hai: 'Sir, Failure Mode and Effects Analysis mein RPN = Severity * Occurrence * Detection hota hai. Hamara baseline average RPN 343 tha. Automated Poka-Yoke gates lagane ke baad Occurrence aur Detection dono drop ho gaye, jisse post-control RPN 14 ho gaya—yani approx 95% risk reduction achieve hui.'

Calculation: Pre-RPN 343 -> Post-RPN 14 (95.9% reduction via automated source inspection).

SLIDE 22: CLOSED-LOOP CONTINUOUS IMPROVEMENT (AUDIT-TO-FMEA)

SLIDE 22

Kya bolna hai: 'Yeh hamara key innovation loop hai: Audit logs sirf history save nahi karte; wo defect signal generate karte hain jo Pareto checklist ko update karta hai, wahan se FMEA RPN recalculate hota hai, aur validation rules refine hoti hain. This is closed-loop Kaizen.'

Pipeline: Capture -> Classify -> Quantify -> Prioritize -> Improve -> Verify.

SLIDE 23: DEMING'S PDCA CONTINUOUS IMPROVEMENT CYCLE

SLIDE 23

Kya bolna hai: 'Poora software development Deming ke PDCA cycle par structured hai:

- Plan: Baseline defects aur CTQ targets identify kiye.
- Do: Poka-Yoke aur CustomTkinter UI build kiya.
- Check: 42 automated tests aur CTQ metrics verify kiye.
- Act: Validation rules standardize kiye aur clean documentation freeze ki.'

SLIDE 24: AUTOMATED QUALITY ASSURANCE & GOLDEN VIVA TAKEAWAY

SLIDE 24

Kya bolna hai: 'Sir, conclusion mein, hamara system 42 out of 42 tests passing ke sath 0.53 seconds mein verify hota hai.

And to conclude with my Golden Viva statement:

"Quality Goal Q07 was achieved not merely by building a data entry screen, but by engineering a proactive Total Quality Management system: Poka-Yoke validation guarantees zero defect entry at the gate, transactional SQLite persistence guarantees atomic integrity, and automated statistical analysis transforms raw audit trails into continuous organizational learning." Thank you, Sir.'

PART 4: TOP 7 VIVA COUNTER-QUESTIONS & WINNING ANSWERS

Critical counter-questions professors frequently ask with exact responses.

Q1: "Yeh toh normal inventory software lag raha hai, isme TQM kahan hai?"

Sir, normal software reactive hota hai—wo ghalat data accept kar leta hai aur baad mein log report check karke manually theek karte hain.

Hamare system mein TQM ke 4 core pillars embedded hain:

1. Poka-Yoke (Deming Principle 3): Entry gate par hi defect prevention.

2. Statistical Process Control: Real-time Pareto (80/20) aur dynamic CTQ Scorecard.

3. FMEA Risk Mitigation: Mathematically RPN ko 343 se 14 tak reduce karna.

4. Kaizen Feedback Loop: Audit logs passive files nahi hain, wo live defect metrics ban kar FMEA ko feed karti hain.

[RULE/FORMULA]: TQM is built into the architecture, not added as a post-hoc report.

Q2: "Agar user category mein spelling ghalat daale ya negative quantity daale toh kya hoga?"

Sir, user manually category type kar hi nahi sakta kyunki humne Controlled Dropdown Taxonomy di hai.

Aur agar user quantity mein negative number daalta hai, toh pehle UI validator use rokta hai, phir domain service reject karti hai, aur third line of defense mein SQLite ka CHECK(quantity >= 0) constraint database level par commit ko rollback kar deta hai.

Invalid data database tak pahunch hi nahi sakta under any condition.

[RULE/FORMULA]: 3-Tier Defense: UI Regex/Mask -> Python Domain Service -> SQLite Constraints.

Q3: "Deming ka Principle 3 'Cease dependence on mass inspection' yahan kaise fit baithta hai?"

Sir, mass inspection ka matlab hota hai maheene ke aakhir mein physical warehouse audit karke dhoondhna ki stock kahan ghalat hua.

Deming ne kaha tha ki quality process ke andar build karo taaki inspection ki zaroorat hi na pade.

Hamara Poka-Yoke validation har single entry ko gate par verify karta hai, isliye retrospective mass correction ki zaroorat khatam ho jaati hai.

[RULE/FORMULA]: Deming Principle 3 = Source inspection at the point of creation.

Q4: "FMEA mein RPN 343 se 14 kaise hua? Formula aur calculation samjhao."

Sir, FMEA formula hai: RPN = Severity (S) * Occurrence (O) * Detection (D).

• Malformed SKU ka initial score tha: S=7, O=7, D=7 => RPN = 343. Yahan detection manual checking thi isliye score high (poor) tha.

• Humne automated regex check aur unique index lagaya. Isse Occurrence 7 se drop hokar 1 ho gayi, aur Detection instant ho gayi (D=1).

• Naya score: 7 * 1 * 1 = 7 (overall average dropped from 343 to 14), which is a 95.9% risk reduction.

[RULE/FORMULA]: RPN = Severity x Occurrence x Detection (Target: Lowest RPN through Poka-Yoke).

Q5: "SQLite kyun use kiya? Agar app achanak crash ho jaaye toh audit log aadha toh nahi bachega?"

Sir, SQLite in-process, zero-configuration aur lightweight hai jo university desktop evaluation ke liye ideal hai.

Data safety ke liye humne Write-Ahead Logging (WAL) enable kiya hai aur har operation atomic transaction (BEGIN TRANSACTION ... COMMIT) ke andar hota hai.

Product update aur audit log entry dono ek hi transaction mein commit hote hain. Agar power cut ya crash hua, toh SQLite automatic rollback kar deta hai; aadha record kabhi commit nahi ho sakta.

[RULE/FORMULA]: ACID Transactions: All-or-nothing guarantee via SQLite WAL Mode.

Q6: "Pareto chart mein 'Vital Few' ka kya relevance hai?"

Sir, Juran ke 80/20 rule ke mutabiq, 80% problems sirf 20% causes ki wajah se aati hain.

Hamare Pareto chart mein Missing Fields (64) aur Invalid Price Formatting (42) milkar 67.9% errors create kar rahe the.

Evaluators ko yeh dikhata hai ki management ko saare 100 features par time waste karne ki jagah pehle in top 2 vital few failure modes par Poka-Yoke lagana chahiye tha, jo humne effectively kiya.

[RULE/FORMULA]: Cumulative % = (Sum Defects_1..k / Total Defects) * 100.

Q7: "Architecture mein 'Screen Never Writes SQL' kyun zaroori hai?"

Sir, agar UI screens direct SQL queries likhein, toh data validation bypass ho sakti hai aur code testable nahi rehta.

Hamare 3-tier architecture mein GUI view sirf ProductService ko call karta hai, jahan business rules check hote hain, aur persistence sirf ProductRepository handle karta hai.

Isi wajah se hum 42/42 pytest tests bina UI open kiye headless mode mein 0.53 seconds mein verify kar paate hain.

[RULE/FORMULA]: Separation of Concerns: Views -> Services/Validators -> Repositories -> SQLite.